Web Accessibility (a11y) Checklist

Comprehensive checklist combining web.dev Learn Accessibility course topics with WCAG 2.2, mobile, cognitive, and organizational best practices.

Sources: Items sourced from the web.dev Learn Accessibility articles are labeled by module name. Items labeled *"WCAG 2.2"* come from the new WCAG 2.2 success criteria. Items labeled *"Beyond articles"* are additional recommendations outside the web.dev course.

HIGH PRIORITY

Address these first. They represent the most impactful and commonly tested accessibility requirements.

☐	Checklist Item	Source
Semantic HTML & ARIA		
☐	Use semantic HTML elements (button, nav, main, etc.) instead of generic divs/spans with ARIA roles	ARIA and HTML
☐	Follow the five rules of ARIA: prefer native HTML; don't change native semantics; all interactive ARIA controls must be keyboard-usable; don't use role="presentation" or aria-hidden on focusable elements; all interactive elements must have accessible names	ARIA and HTML
☐	Add ARIA roles, states, and properties only when no native HTML equivalent exists	ARIA and HTML
☐	Ensure every interactive element has an accessible name (via label, aria-label, or aria-labelledby)	ARIA and HTML
Content Structure & Landmarks		
☐	Use landmark elements (header, nav, main, aside, footer) to define page regions	Content Structure
☐	Provide at least one landmark per page	Content Structure
☐	Use a logical heading hierarchy (h1–h6) without skipping levels	Content Structure
☐	Don't use headings purely for visual styling; use CSS instead	Content Structure
☐	Use ordered, unordered, and description lists with proper HTML elements	Content Structure
The Document		
☐	Add a unique, descriptive <title> to every page, front-loading the most specific content	The Document

☐	Checklist Item	Source
☐	Set the lang attribute on the <html> element with a valid language code	The Document
☐	Mark sections in a different language with their own lang attribute	The Document
☐	Add a skip navigation link as the first focusable element on each page	The Document
Keyboard Focus		
☐	Ensure all interactive elements are reachable and operable via keyboard alone	Keyboard Focus
☐	Maintain a logical, intuitive focus order that matches the visual layout	Keyboard Focus
☐	Provide visible focus indicators on all focusable elements; never remove outline without a replacement	Keyboard Focus
☐	Use tabindex="0" to add non-native elements to focus order; use tabindex="-1" for programmatic focus only	Keyboard Focus
☐	Never use positive tabindex values (tabindex="1", "2", etc.)	Keyboard Focus
☐	Avoid keyboard traps: users must be able to Tab into and out of every component	Keyboard Focus
JavaScript & Dynamic Content		
☐	Use device-independent event handlers (click, keydown) rather than mouse-only events (mouseover)	JavaScript
☐	Manage focus when content changes dynamically (e.g., modals, route changes, inline edits)	JavaScript
☐	Use aria-live regions to announce dynamic updates to screen readers	JavaScript
☐	Ensure JavaScript-rendered content is represented in the accessibility tree	JavaScript
Images		
☐	Provide descriptive alt text for all informative images	Images
☐	Use empty alt="" for purely decorative images so screen readers skip them	Images
☐	For complex images (charts, infographics), provide a longer text description nearby or via aria-describedby	Images
☐	Avoid embedding essential text inside images; if unavoidable, replicate it in alt text	Images
Color & Contrast		

☐	Checklist Item	Source
☐	Meet WCAG AA minimum contrast ratios: 4.5:1 for normal text, 3:1 for large text	Color & Contrast
☐	Ensure non-text UI components (icons, borders, focus rings) meet 3:1 contrast against adjacent colors	Color & Contrast
☐	Never use color as the sole means of conveying information (e.g., error states, charts)	Color & Contrast
Animation & Motion		
☐	Respect prefers-reduced-motion media query and reduce/disable non-essential animations	Animation & Motion
☐	Provide a mechanism to pause, stop, or hide any auto-playing content that lasts more than 5 seconds	Animation & Motion
☐	Avoid content that flashes more than 3 times per second	Animation & Motion
Forms		
☐	Associate every input with a visible <label> using for/id pairing	Forms
☐	Group related fields using <fieldset> and <legend>	Forms
☐	Provide clear, programmatically-associated error messages that describe what went wrong and how to fix it	Forms
☐	Use autocomplete attributes on common input fields (name, email, address, etc.)	Forms
☐	Ensure form validation messages are announced to screen readers (via aria-live or aria-describedby)	Forms
☐	Don't rely on placeholder text as a substitute for labels	Forms
Patterns, Components & Design Systems		
☐	Follow WAI-ARIA Authoring Practices for custom widgets (tabs, menus, dialogs, accordions, etc.)	Patterns
☐	Implement proper modal dialog behavior: trap focus inside, return focus on close, use role="dialog" and aria-modal	Patterns
☐	Use established accessible component libraries or thoroughly test custom components	Patterns
Testing — Automated		
☐	Run automated a11y scans (axe, Lighthouse, WAVE) on every page template	Automated Testing
☐	Integrate automated accessibility checks into CI/CD pipeline	Beyond articles
☐	Understand automated tools catch only ~30–50% of issues; don't treat a clean scan as fully accessible	Automated Testing

☐	Checklist Item	Source
Testing — Manual		
☐	Tab through every page using keyboard only; verify all interactions work without a mouse	Manual Testing
☐	Test with browser zoom at 200% and 400% — ensure no content is cut off or overlapping	Manual Testing
☐	Verify reading order makes sense when CSS is disabled	Manual Testing
☐	Check all pages in high-contrast mode	Manual Testing
Testing — Assistive Technology		
☐	Test with at least one screen reader (NVDA on Windows, VoiceOver on macOS/iOS, TalkBack on Android)	AT Testing
☐	Verify all interactive elements are announced with correct role, name, and state	AT Testing
☐	Test critical user flows end-to-end with a screen reader	AT Testing
WCAG 2.2 New Criteria (Level AA)		
☐	Focus Not Obscured (2.4.11): Ensure focused elements are not completely hidden by sticky headers, footers, or cookie banners	WCAG 2.2
☐	Dragging Movements (2.5.7): Provide single-pointer alternatives for any drag-and-drop functionality	WCAG 2.2
☐	Target Size Minimum (2.5.8): Ensure touch/click targets are at least 24×24 CSS pixels, with exceptions for inline text links	WCAG 2.2
☐	Consistent Help (3.2.6): Place help mechanisms (contact info, chat, FAQ) in the same relative location across all pages	WCAG 2.2
☐	Accessible Authentication (3.3.8): Don't require cognitive function tests (memorizing passwords, solving puzzles) as the sole login method	WCAG 2.2
☐	Redundant Entry (3.3.9): Don't ask users to re-enter information they've already provided in the same session	WCAG 2.2

MEDIUM PRIORITY

Tackle these once the high-priority fundamentals are solid. They cover specialized content types, mobile, cognitive needs, and process maturity.

☐	Checklist Item	Source
Video & Audio		
☐	Provide synchronized captions for all pre-recorded video content	Beyond articles
☐	Provide transcripts for all audio-only content (podcasts, audio clips)	Beyond articles
☐	Provide audio descriptions for video where important visual information isn't conveyed by audio alone	Beyond articles
☐	Ensure media players are fully keyboard-accessible with visible controls	Beyond articles
Typography & Readability		
☐	Use relative units (rem, em, %) for font sizes so text scales with user preferences	Beyond articles
☐	Ensure line height is at least 1.5× font size for body text	Beyond articles
☐	Ensure paragraph spacing is at least 2× font size	Beyond articles
☐	Don't disable user text resizing (avoid maximum-scale=1 in viewport meta tag)	Beyond articles
Mobile Accessibility		
☐	Support both portrait and landscape orientations unless a specific orientation is essential	Beyond articles
☐	Ensure touch targets are at least 44×44 CSS pixels (iOS) / 48×48dp (Android) for comfortable tapping	Beyond articles
☐	Don't disable pinch-to-zoom (avoid user-scalable=no)	Beyond articles
☐	Test gesture-based interactions with assistive tech (VoiceOver gestures, TalkBack swipes)	Beyond articles
☐	Ensure custom gestures (swipe, long press) have single-tap alternatives	Beyond articles
Cognitive Accessibility		
☐	Write in plain, clear language appropriate to your audience	Beyond articles
☐	Use consistent navigation and UI patterns across the site	Beyond articles
☐	Break long content into scannable chunks with meaningful headings	Beyond articles
☐	Provide clear, specific instructions before complex tasks (forms, multi-step processes)	Beyond articles

☐	Checklist Item	Source
☐	Allow users enough time to complete tasks; warn before timeouts and offer extensions	Beyond articles
SPA (Single-Page App) Patterns		
☐	Announce route changes to screen readers (e.g., update document.title and move focus to main content)	Beyond articles
☐	Manage scroll position on navigation so users aren't stranded mid-page	Beyond articles
☐	Ensure browser back button works predictably in client-side routing	Beyond articles
☐	Use aria-live regions for in-page loading states and content updates	Beyond articles
Document & PDF Accessibility		
☐	Tag PDFs with proper structure (headings, lists, tables, reading order)	Beyond articles
☐	Add alt text to images within PDFs and documents	Beyond articles
☐	Ensure downloadable documents (Word, PDF) pass accessibility checks before publishing	Beyond articles
☐	Provide HTML alternatives for critical PDF content when possible	Beyond articles
Tables & Data		
☐	Use <caption> elements to describe the purpose of data tables	Beyond articles
☐	Use <th> with scope attributes to define row and column headers	Beyond articles
☐	Don't use tables for layout; use CSS Grid or Flexbox instead	Beyond articles
☐	For complex tables, use headers/id attributes to associate data cells with headers	Beyond articles
Testing Pipeline		
☐	Add eslint-plugin-jsx-a11y (or framework equivalent) to your linting configuration	Beyond articles
☐	Include accessibility acceptance criteria in user stories and QA checklists	Beyond articles
☐	Schedule periodic manual audits (quarterly or per release cycle)	Beyond articles
☐	Conduct usability testing with people who use assistive technologies	Beyond articles
Legal & Compliance Awareness		
☐	Identify which accessibility standard applies to your project (WCAG 2.1 AA, WCAG 2.2 AA, EN 301 549, Section 508)	Beyond articles

☐	Checklist Item	Source
☐	Publish an accessibility statement on your site describing conformance level and contact info for issues	Beyond articles
☐	Review applicable regulations: ADA Title II/III, European Accessibility Act, state-level laws	Beyond articles

LOW PRIORITY

Valuable but more specialized or forward-looking. Address these as your accessibility program matures.

☐	Checklist Item	Source
WCAG 2.2 Level AAA Criteria		
☐	Focus Not Obscured Enhanced (2.4.12): Ensure focused elements are never even partially hidden by other content	WCAG 2.2
☐	Focus Appearance (2.4.13): Provide focus indicators that meet minimum size (2px perimeter) and contrast requirements	WCAG 2.2
☐	Accessible Authentication Enhanced (3.3.9): Remove all cognitive tests from auth, including object recognition and personal content	WCAG 2.2
Internationalization & Language		
☐	Support right-to-left (RTL) layouts for languages that require it	Beyond articles
☐	Test screen reader pronunciation when switching between languages on the same page	Beyond articles
☐	Ensure translated content maintains the same structural accessibility as the original	Beyond articles
Accessible Data Visualization		
☐	Provide text-based alternatives (tables, summaries) for charts and graphs	Beyond articles
☐	Use patterns, textures, or labels in addition to color in charts	Beyond articles
☐	Make interactive data visualizations keyboard-navigable with ARIA	Beyond articles
Accessibility Overlays		
☐	Understand that overlay widgets don't achieve WCAG conformance and can introduce new barriers	Beyond articles
☐	Advocate for native, built-in accessibility over third-party overlay tools	Beyond articles
Procurement & VPATs		
☐	Request Voluntary Product Accessibility Templates (VPATs) when evaluating third-party tools	Beyond articles
☐	Include accessibility requirements in vendor contracts and RFPs	Beyond articles
WCAG 3.0 Awareness		
☐	Monitor W3C's WCAG 3.0 (Silver) development for the upcoming scoring-based conformance model	Beyond articles

☐	Checklist Item	Source
☐	Note that WCAG 3.0 is not yet finalized and is not actionable for current compliance	Beyond articles
Organizational Practices		
☐	Provide accessibility training for designers, developers, content authors, and QA	Beyond articles
☐	Designate an accessibility champion or team to maintain standards over time	Beyond articles
☐	Embed accessibility into design reviews and code review checklists	Beyond articles

Generated March 2026. Based on web.dev Learn Accessibility course, WCAG 2.2 (W3C), Web Almanac 2025, and industry best practices.